Public Safety Alerts Aggregator & Explorer

Team Name: Critical Signal

Course: ICT303 / ICT306

Team Members:

- Ettore Vescio - 11799221 - Team Leader
- Thomas Smith - 11787500 - Deputy Team Leader
- Christian Finocchiaro - 11854560
- Pradip Pandey - 11831987

Approach/Methodology Justification & Alignment

Ettore Vescio

Problem Statement & Solution Overview

Problem Statement

- Public safety alerts are distributed across multiple government sources
- Data formats vary between agencies (RSS, APIs, JSON structures)
- Users must manually check multiple websites
- Existing systems can be difficult to filter or interpret quickly during emergencies

Proposed Solution

Critical Signal is a centralised public safety alert aggregation platform that:

- Collects alerts from multiple NSW government sources
- Standardises and stores alert data
- Displays alerts on an interactive map
- Supports filtering and subscriptions
- Provides a single, user-friendly interface for situational awareness

Approach / Methodology Justification & Alignment

Chosen Methodology

Agile Scrum with Iterative Prototyping

Why Agile Was Selected

- Requirements evolved during development
- External APIs and feeds were inconsistent
- Early proof-of-concepts reduced technical risk
- Continuous testing improved architecture decisions
- Short iterations supported rapid refinement

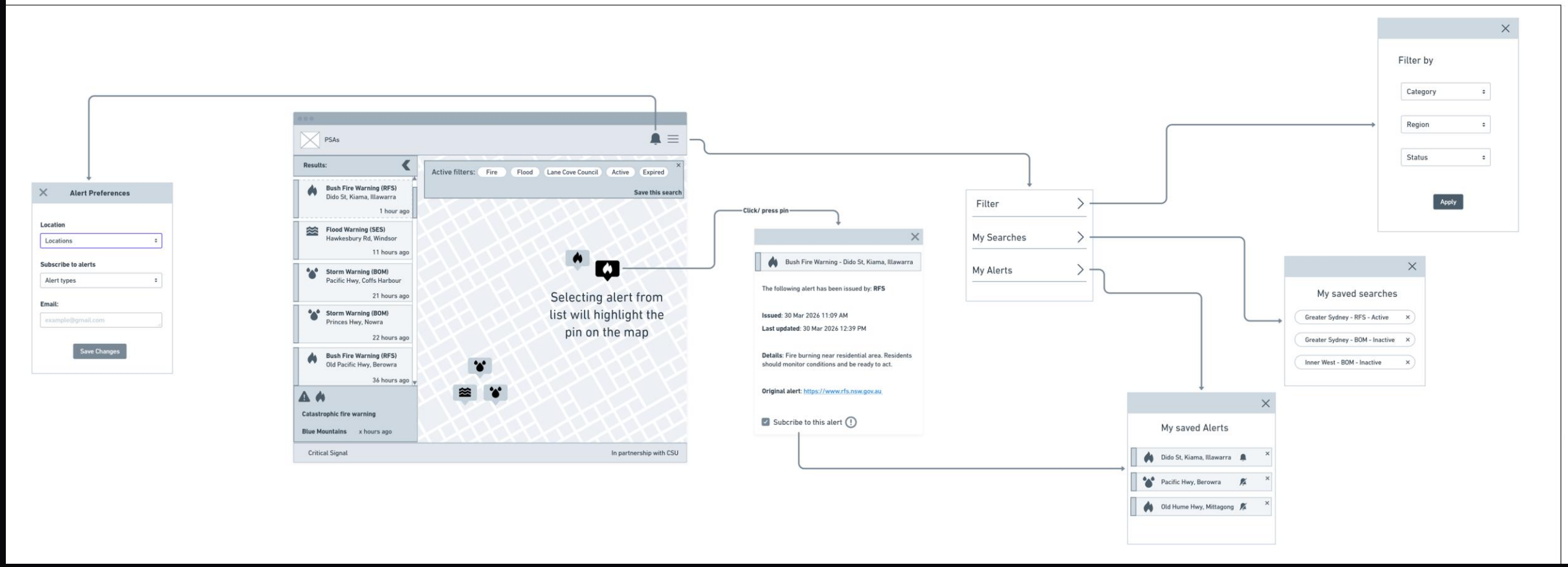
Milestone / Deliverable	Key Tasks	Due Date	Owner(s)
Sprint 1 (Core Backend)	Database setup, ingestion script, basic API endpoints	22/05/2026	Ettore Vescio Thomas Smith
Sprint 2 (API & frontend prototype)	Implement filtering (region/ category), basic frontend	29/05/2026	Christian Finocchiaro Pradip Pandey
Sprint 3 (Notifications & optimisation)	Implement email notifications, subscription logic, and frontend integration	05/06/2026	Whole team
Sprint 4 (Validation & Demonstration)	Validation and testing for frontend and backend logic.	12/06/2026	Whole team

Software Solution Design

Thomas Smith

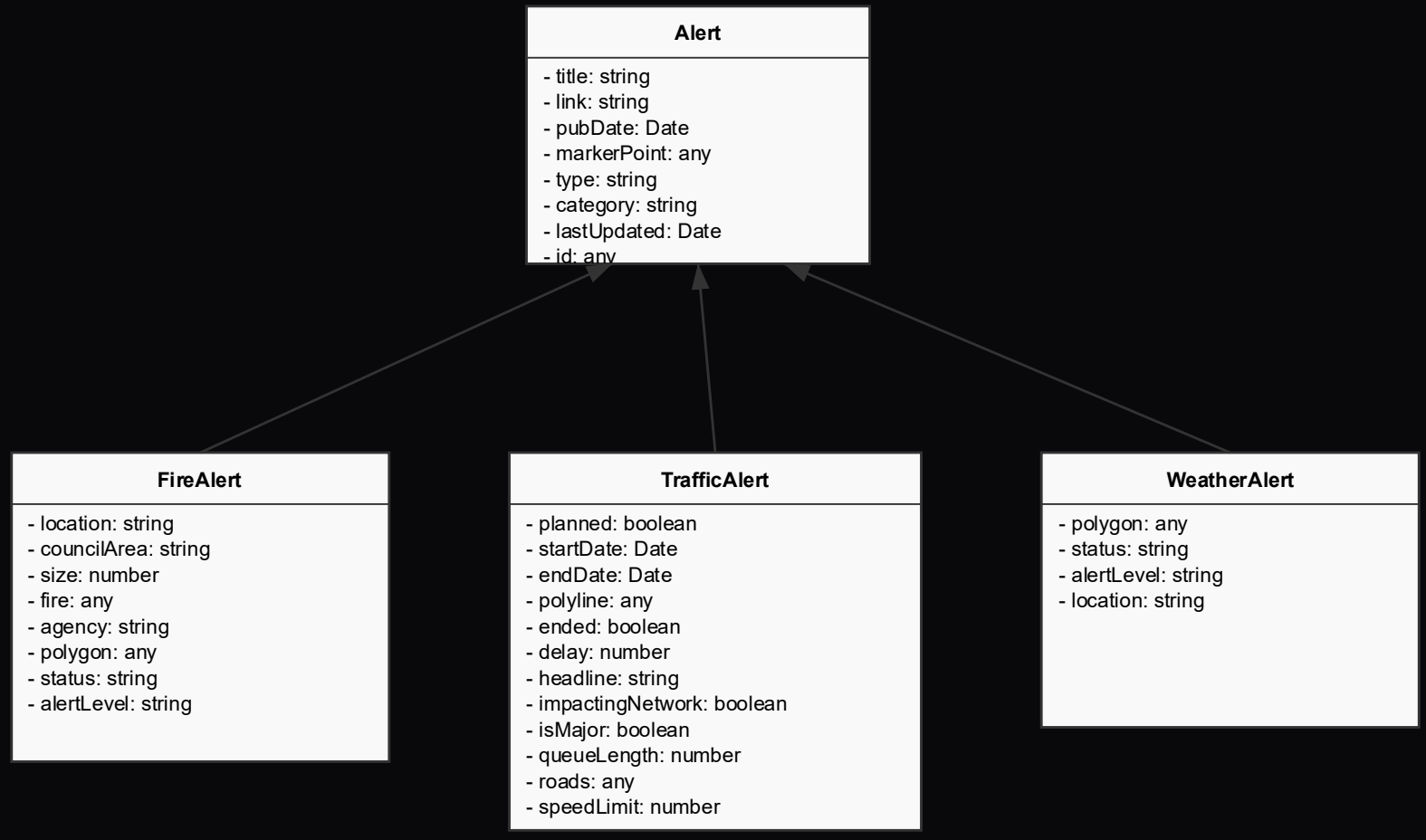
Ui/UX Wireframes

The following wireframes demonstrate the initial UI/UX plan for the frontend interface. It shows how the pages elements will be laid out and how the user can interact with the page.



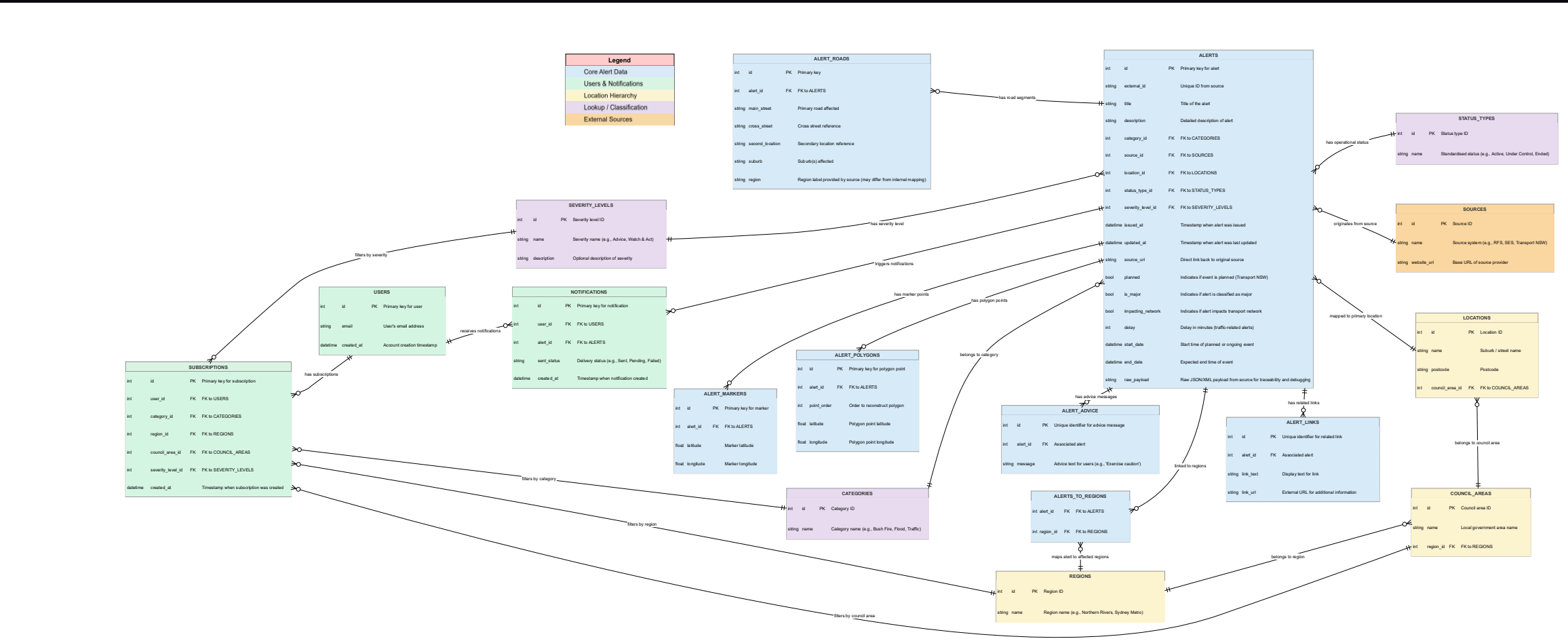
Class Diagram

This diagram shows the planned structure and inheritance of the alerts classes. This class will be used to create objects that will store the alert and will be used when transporting the alerts from the backend to the frontend.



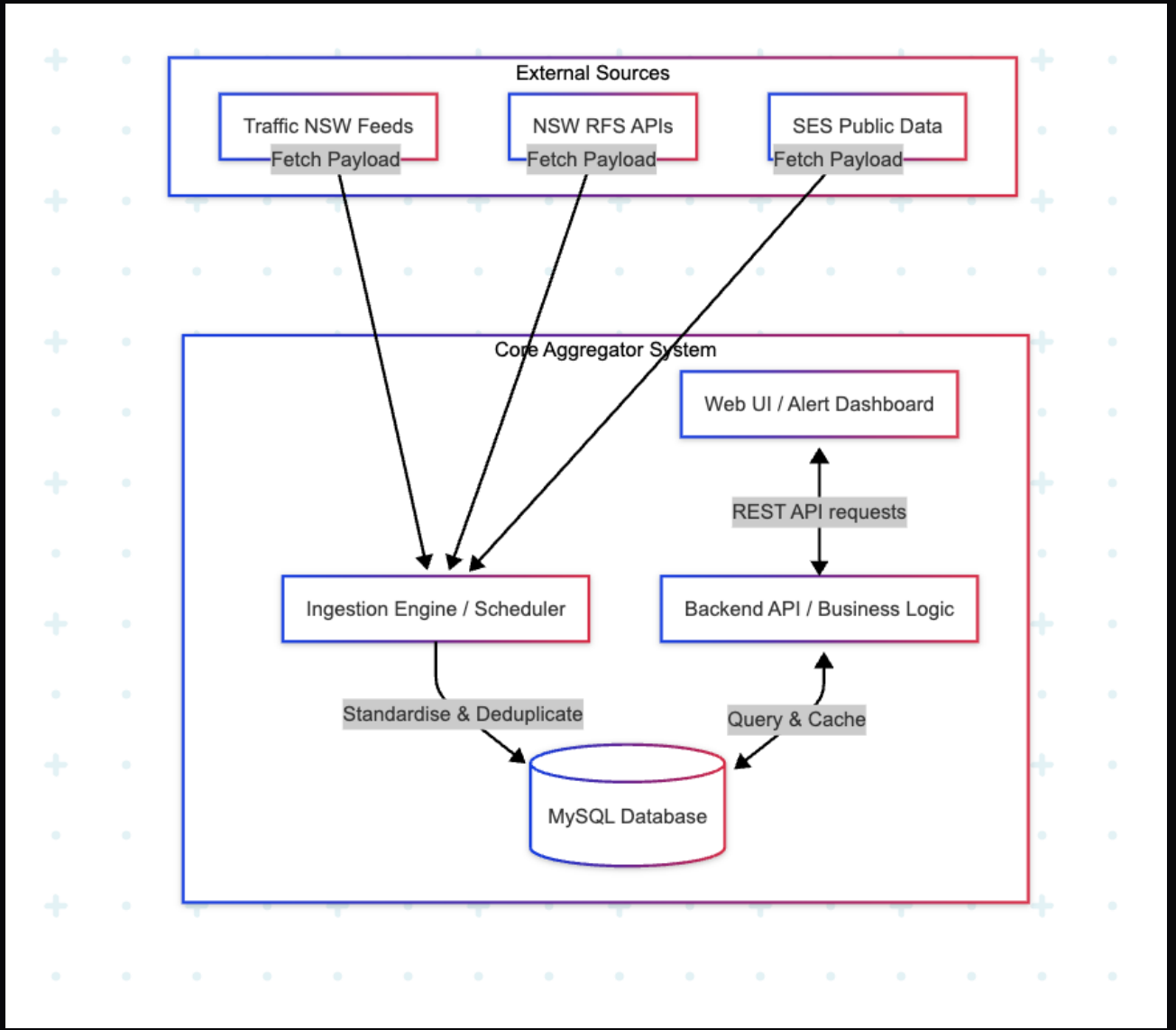
Entity Relationship Diagram

This diagram describes the structures and relationships involved in the database system for this project.



System Architecture

This diagram shows the flow of data through the whole system. From the warning feeds to the frontend, it outlines the journey that data takes from its external sources, to the database, then to the frontend interface.

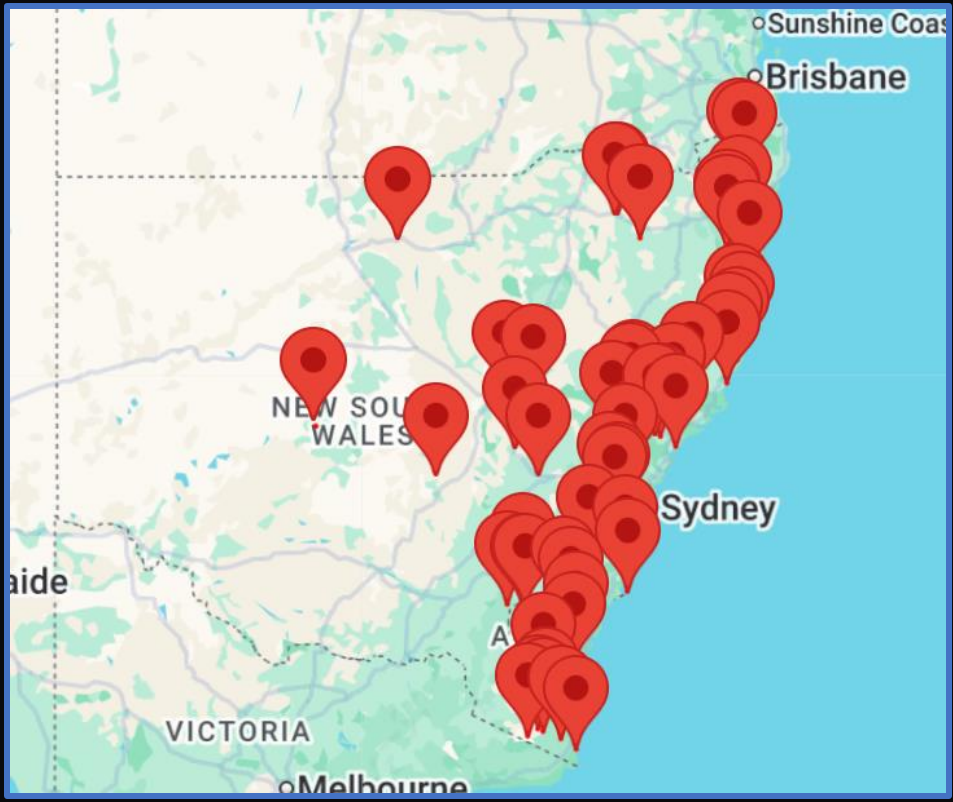


Key Design Decisions

Christian Finocchiaro

Decision 1: Google Maps (Mapping Library)

"Our system must accurately show where incidents are occurring by placing pins on a live map, updated in real-time, as well as reflect the latest information for existing alerts"



- Australian users are familiar with Google Maps
- Thorough documentation for developers
  - Provided by Google
- Built-in functionality satisfies our project requirements
  - Validated through proof-of-concept testing
- Free tier (28,500 loads/month) is sufficient for the project's scale

Decision 2: Prioritised Traffic Alerts Over Weather Alerts

"Our system must ingest and display alerts from at least three different sources (government agencies)"

- Confirmed Sources:
  - NSW Rural Fire Service (RFS)
  - NSW State Emergency Service (SES)
  - Transport for NSW (TFNSW)
- The problem?
  - Not enough active alerts
    - The system must display meaningful data & preserve a level of trust from our users
- Our solution
  - Substitute BOM for TFNSW
    - Provides a larger quantity of alerts
    - Satisfies the three-source requirement

The BOM can be included as a source in a future update. Our prototype needs reliable data.

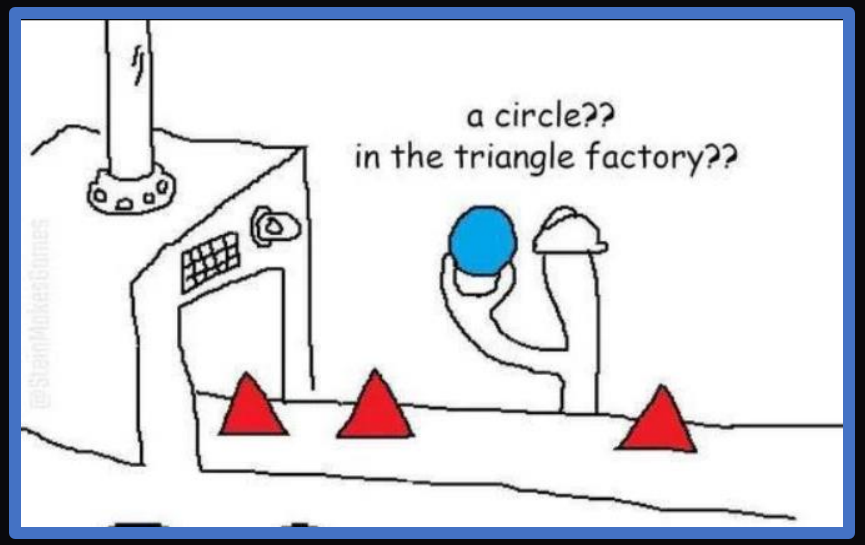
Decision 3: SQLite for development & MySQL for full release

Both free, well-documented, and familiar to the team

- Runs locally (no server needed)
- Team member can work independently
- Can be transitioned into the full-release if needed

- Supports multiple concurrent users
- Scales with more users, alerts, and data sources
- Supports our database structure (relational)

Decision 4: Data needs to be normalized/ standardised



- Each source structures and labels its data differently
  - Alerts from the same source require standardisation before being stored
  - Some fields are common between sources (e.g. Location)
  - Other fields will source specific (e.g. Fire has a size field)

- Useful for:
  - Consistently rendering alerts on the front end
  - Accurate alert filtering (frontend) & querying (database)
  - Displaying relevant data to the user

Data / Evidence / Operational Considerations

Ettore Vescio

Data / Evidence / Operational Considerations

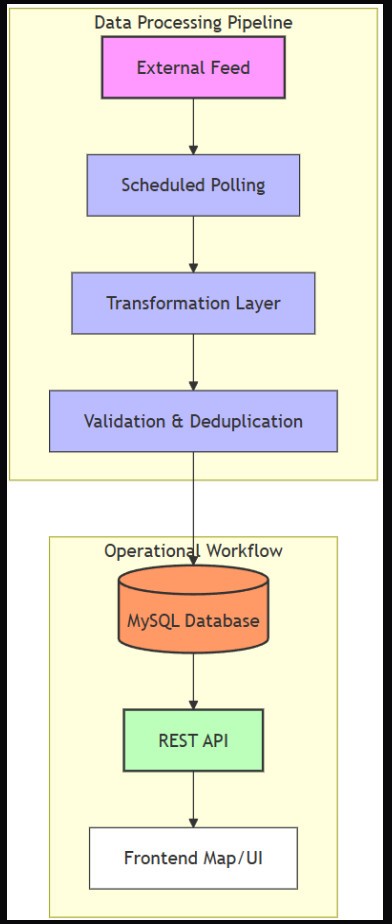
External Data Sources

- NSW Rural Fire Service (RSS)
- Transport for NSW API
- NSW SES feeds
- Future extensibility for additional Australian sources

Key Data Considerations

- Heterogeneous source formats
- Geospatial data support:
  - markers
  - polygons
  - polylines
- Duplicate alert prevention using external identifiers
- Raw payload storage for debugging and traceability

Operational Workflow



Operational & Privacy Controls

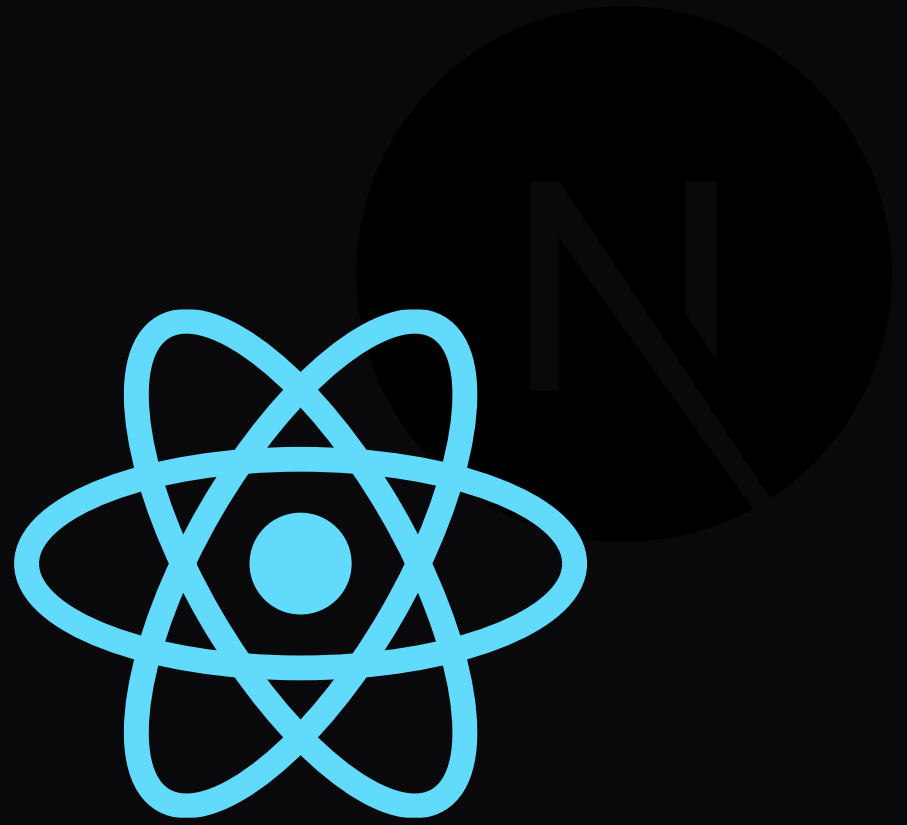
- HTTPS/TLS encryption
- Minimal personal data storage
- Graceful handling of unavailable APIs
- Scheduled ingestion tasks
- Modular ingestion architecture for maintainability

Technology Stack

Thomas Smith

Frontend

The frontend will be powered by Next.js which utilises React.js framework. For this purpose, the frontend will be developed using Typescript.



- Dynamically generated and responsive design
- Modular, component-based structure
- Google Maps compatible
- Modern, secure and professional
- Built-in routing
- Built-in JSON compatibility
- Well documented

Backend / API

The backend API will be developed in Node.js with the Express.js framework. This is an industry standard framework for easy, intuitive and modular APIs.

- Modular
- Well documented
- Built-in JSON compatibility
- Package manager
- Customisable
- Professional

Database

The database will be built using SQLite. This is a Structured Query Language (SQL) database that is hosted within the backend eliminating the need for an externally hosted SQL database. The final release may transition to MySQL depending on client requirements.

- Structured
- Relational
- Self hosted
- Lightweight
- Trusted node.js packages

Version Control

This project will utilise Git and GitHub to handle version control, collaboration, project storage and backup.

- Branches
- Remote access
- Security scans
- Compatible with all devices
- Industry standard
- Peer review before merging with higher branches

Non-Functional Requirements: Security, Performance, Reliability

Pradip Pandey

Non-Functional Requirements

01 SECURITY THREATS

SQLi Prevention

SECURE-BY-DESIGN

XSS & CORS Mitigation

THREAT NEUTRALIZATION

02 PERFORMANCE & STABILITY

Concurrency & Speed

SQLITE OPTIMIZATION

Fault Tolerance

FAULT-TOLERANT INGESTION

03 COMPLIANCE & PRIVACY

Privacy Act 1988

AU COMPLIANCE

Data Protection

ZERO PII STORAGE

Cybersecurity & Risk Controls

INJECTION & CORS

SQLi prevention via DAO placeholder parameters. CORS middleware restricts API requests strictly to authorized client URLs.

XSS MITIGATION

React 19 native HTML escaping combined with strict content validation and sanitization for dynamic description fields.

SECRET MANAGEMENT

Isolation of API keys in local environment files, strictly excluded from version control via security protocols.

Performance & Reliability

SYSTEM OPTIMIZATION

- **Database Concurrency:** WAL mode permits simultaneous read operations during active ingestion cycles.
- **Geospatial Indexes:** Sub-millisecond response times for category and temporal queries.
- **Fault-Tolerance:** IngestOrchestrator handles external failures gracefully with cached data fallbacks.
- **Modular Architecture:** Clean separation between Services, DAOs, and Controllers.

Compliance & Privacy

DATA MINIMISATION

Aligned with the Australian Privacy Act 1988. Physical addresses, names, and phone numbers are intentionally excluded from collection.

ZERO TRACKING DATA

Storage is limited to email addresses and hashed/salted preference records. Compromise would expose zero PII or geographic tracking data.

Demonstrations

Evidence of Testing

Evidence of Testing – Frontend Acceptance Checklist

Frontend Acceptance Checklist

Navigation

☒ Clicking the hamburger menu icon opens and closes the menu drawer

☒ Clicking the bell icon opens the alert preferences modal

☒ Clicking outside the alert preferences modal closes it

Menu Drawer

☐ Filter, My Searches and My alerts buttons are visible and can be clicked

☒ The menu drawer closes when the ✕ button is clicked

☒ Clicking outside the menu drawer closes it

Alert List

☒ All alert cards render without errors

☒ The correct information is shown on each alert card

☒ The alert list correctly scrolls when content overflows the container

☐ Clicking an alert card pans and zooms the map to the corresponding pin

☐ Clicking the highlighted pin opens the modal showing more information about the alert (including source link)

Alert Preferences Modal

☒ Modal appears when the bell icon is clicked

☐ The service dropdown shows all available alert sources

Summary of Prototype Testing

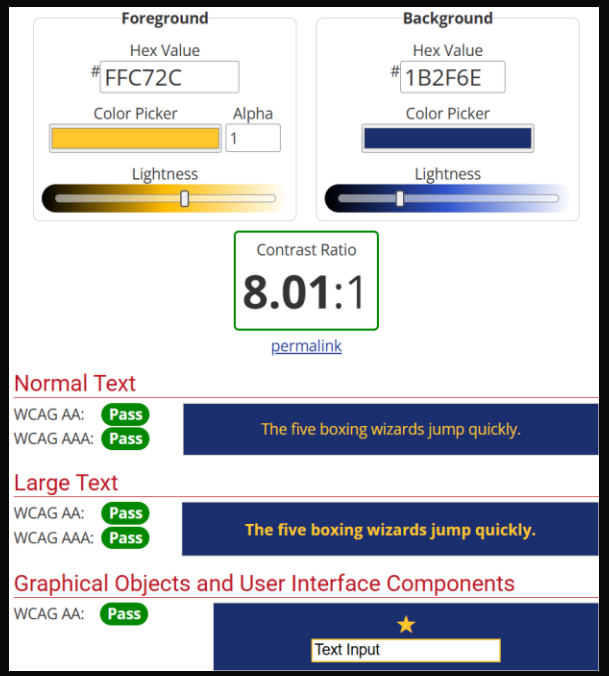
- Unchecked items represent features planned for session 2
  - At this stage of development, the project pipeline (frontend, backend, database) is not connected
- Checked items have been validated and tested in the current prototype

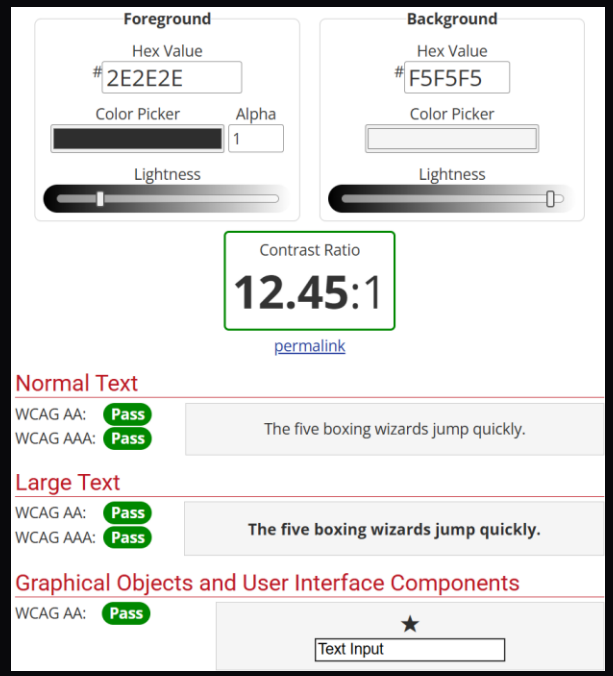
Requirements Being Addressed

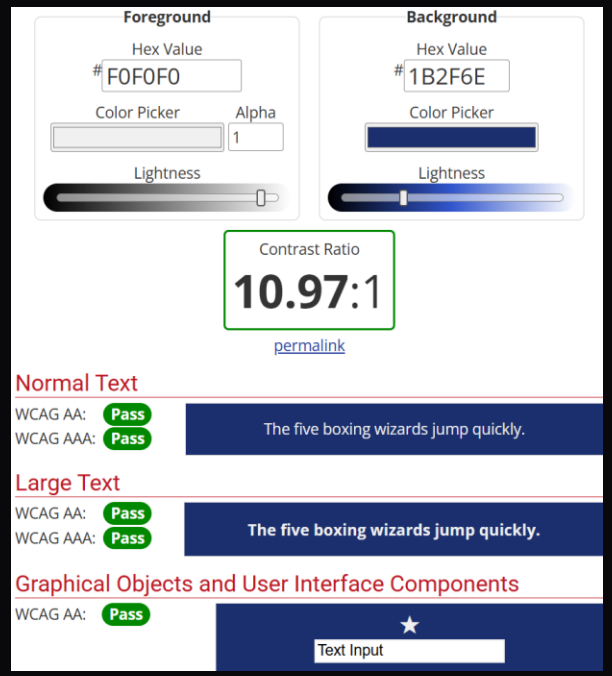
1. **R01** — intuitive map-based interface
2. **R03** — filter by region/category/severity
   a. Partially covered by testing but requires a backend connection to fully implement
3. **R06** — clicking an alert shows detail including source link and location
4. **R08** — accessible on all devices, clear responsiveness → covered by your accessibility section

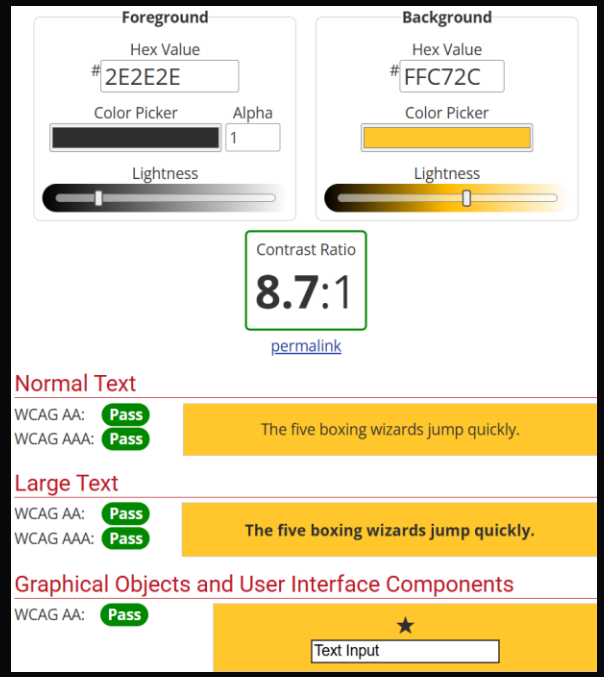
Full file is available

Evidence of Testing – Frontend WCAG AA/AAA









Evidence of Testing – Unit Tests

```
(node:12512) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/services/IngestOrchestratorService.test.js
 PASS  tests/normalizers/rfsNormalizer.test.js
 PASS  tests/services/AlertPersistenceService.test.js
 PASS  tests/transformers/dateTransformer.test.js
 PASS  tests/transformers/locationTransformer.test.js
 PASS  tests/normalizers/tfnswNormalizer.test.js
 PASS  tests/transformers/statusTransformer.test.js
 PASS  tests/transformers/severityTransformer.test.js
 PASS  tests/transformers/categoryTransformer.test.js
 PASS  tests/collectors/rfsCollector.test.js
 PASS  tests/collectors/tfnswCollector.test.js

Test Suites: 11 passed, 11 total
Tests:       169 passed, 169 total
Snapshots:   0 total
Time:        1.293 s
Ran all test suites.
```

Evidence of Testing – API endpoint /alerts

GET localhost:3001/alerts

Status: 200 OK Size: 1.66 MB Time: 65 ms


```
[
  {
    "title": "Beasleys Road, Towamba State Forest",
    "link": "https://www.rfs.nsw.gov.au/fire-information/fires-near-me",
    "pubDate": "2026-05-23T19:27:00.000Z",
    "markerPoint": {
      "lat": -37.11162729199998,
      "lng": 149.681506183
    },
    "type": "Fire",
    "category": "Hazard Reduction",
    "lastUpdated": "2026-05-24T05:27:00.000Z",
    "externalID": "651687",
    "rawData": {
      "type": "Feature",
      "geometry": {
```

Evidence of Testing – API endpoint /alerts/traffic

GET localhost:3001/alerts/traffic

Status: 200 OK Size: 1.57 MB Time: 41 ms


```
[
  {
    "title": "CHANGED TRAFFIC CONDITIONS Mamre Road upgrade",
    "link": "https://www.livetraffic.com/incident-details/212927",
    "pubDate": "2024-11-01T04:26:14.000Z",
    "markerPoint": {
      "lat": -33.7913423,
      "lng": 150.7698515
    },
    "type": "Traffic",
    "category": "CHANGED TRAFFIC CONDITIONS",
    "lastUpdated": "2024-11-08T04:13:49.940Z",
    "externalID": 212927,
    "rawData": {
      "type": "Feature",
      "id": 212927,
```

Evidence of Testing – API endpoint /alerts/fire

GET localhost:3001/alerts/fire

Status: 200 OK Size: 93.69 KB Time: 29 ms


```
[
  {
    "title": "Beasleys Road, Towamba State Forest",
    "link": "https://www.rfs.nsw.gov.au/fire-information/fires-near-me",
    "pubDate": "2026-05-23T19:27:00.000Z",
    "markerPoint": {
      "lat": -37.11162729199998,
      "lng": 149.681506183
    },
    "type": "Fire",
    "category": "Hazard Reduction",
    "lastUpdated": "2026-05-24T05:27:00.000Z",
    "externalID": "651687",
    "rawData": {
      "type": "Feature",
      "geometry": {
```

Evidence of Testing – API endpoint /alerts/{id}

GET localhost:3001/alerts/651687

Status: 200 OK Size: 13.03 KB Time: 110 ms


```
{
  "title": "Beasleys Road, Towamba State Forest",
  "link": "https://www.rfs.nsw.gov.au/fire-information/fires-near-me",
  "pubDate": "2026-05-23T19:27:00.000Z",
  "markerPoint": {
    "lat": -37.11162729199998,
    "lng": 149.681506183
  },
  "type": "Fire",
  "category": "Hazard Reduction",
  "lastUpdated": "2026-05-24T05:27:00.000Z",
  "externalID": "651687",
  "rawData": {
    "type": "Feature",
    "geometry": {
      "type": "GeometryCollection",
```

Vulnerability Scan & Remediation Plan | Monitoring & Alerting

OWASP Top 10 findings with remediation | Live observability covering SEC / PERF / RELY / COMP

Vulnerability Scan — OWASP Top 10

MITIGATED PARTIAL LOW RISK = scan result status

A01 LOW Broken Access Control — MITIGATED — SEC

Public read-only dashboard. No auth surface. Email stored as SHA-256 hash + unique token.

Fix: Threat surface does not exist — no action required.

A03 HIGH Injection (SQLi) — MITIGATED — SEC

All 4 DAOs use db.prepare(sql).run(...params). User input never concatenated.

Fix: Tested: ' OR 1=1-- payloads across all endpoints → 0 rows returned.

A05 MEDIUM Security Misconfiguration — MITIGATED — SEC

CORS origin whitelist. API keys in .env + .gitignore. Helmet.js headers enforced.

Fix: Least-privilege: only trusted origin. No NEXT_PUBLIC_ on sensitive keys.

A07 HIGH XSS (Cross-Site Scripting) — MITIGATED — SEC

React 19 auto-escapes JSX. Ingest pipeline strips HTML tags before DB write.

Fix: <script>alert(1)</script> via feed → plain text output, not executed.

A09 MEDIUM Security Logging Failures — PARTIAL — RELY

Ingest failures logged to /backend/logs/ with timestamp + source ID. No auto-alerts yet.

Fix: Session 2: OWASP ZAP + anomaly alerting pipeline closes gap. (Risk R09)

A10 LOW SSRF — LOW RISK — SEC

All external URLs hardcoded in IngestController. No user-supplied URL ever fetched.

Fix: Attack surface does not exist. No further remediation needed.

Monitoring & Alerting — System Observability

LIVE Active in production PLANNED Session 2 roadmap

LIVE RELY Ingest Source Failure

Trigger: TfNSW API returns DNS error or HTTP 5xx

Response: IngestOrchestrator wraps each source in try/catch. Logs timestamp + source ID to /logs/. Other sources unaffected.

LIVE RELY Invalid Coordinate Data

Trigger: markerPoint.lat / .lng is null, NaN, or 0,0

Response: isValidPosition() guard silently skips bad markers. No crash, no user impact. Console warning logged.

LIVE PERF Rate Limit Breach (DoS)

Trigger: Single IP exceeds request threshold per window

Response: Express rate-limiter returns HTTP 429. IP throttled automatically. Server stays responsive under load.

LIVE COMP Data Source Health Check

Trigger: Each source has a last_successful_refresh timestamp

Response: GET /sources/health returns per-source status. Stale sources flagged. Satisfies client SLA (Jason Howarth brief).

PLANNED SEC Anomaly / ZAP Alerting

Trigger: Unusual request volumes or repeated 4xx sequences

Response: Session 2: OWASP ZAP integration + automated alert pipeline. Closes Risk R09 gap for A09 (PARTIAL).

SEC: A03/A07/A05/A10 MITIGATED. RELY: Ingest fault-tolerance + coord guard maintain 99% uptime. PERF: Rate limiter blocks DoS. COMP: Health check satisfies client SLA.

Q & A